TP3 – OpenMP

Introduction to Parallel Programming

Lab Report by Mounia BADDOU

1. Exercise 1 – Hello World with Threads

1.1 Objective

Write an OpenMP program that displays the rank of each thread and the total number of threads used for execution, then test it with different thread counts.

1.2 Implementation

The program uses a `#pragma omp parallel` region so that each thread independently prints its rank using `omp_get_thread_num()`, and one designated thread (rank 0) prints the total count using `omp_get_num_threads()`.

```c
#include <stdio.h>
#include <omp.h>

int main() {
    #pragma omp parallel
    {
        int rank = omp_get_thread_num();
        printf("Hello World! from thread %d\n", rank);

        #pragma omp single
        {
            printf("Parallel execution with %d threads\n",
                   omp_get_num_threads());
        }
    }
    return 0;
}
```

Listing 1: Exercise 1 – Hello World with OpenMP threads

1.3 Output and Analysis

Running with the default thread count (10 threads on the test machine):

```
Hello World! from thread 2
Hello World! from thread 8
Hello World! from thread 4
Hello World! from thread 3
Hello World! from thread 5
Hello World! from thread 7
Hello World! from thread 1
Hello World! from thread 0
Hello World! from thread 9
Hello World! from thread 6
```

Listing 2: Output of Exercise 1 (10 threads)

Running with a user-specified count (5 threads):

```
Enter # of threads: 5
Hello World! from thread 0
```

```
Hello World! from thread 3
Hello World! from thread 2
Hello World! from thread 4
Hello World! from thread 1
```

Listing 3: Output of Exercise 1 (5 threads, interactive input)

1.4 Discussion

- **Non-deterministic order:** The output lines appear in a different order on every run because threads execute concurrently and the OS schedules them independently.
- **Thread count:** The number of threads is controlled either by the `OMP_NUM_THREADS` environment variable, by a call to `omp_set_num_threads()`, or by the `num_threads` clause on the pragma.
- **Shared memory:** All threads share the same address space; the `rank` variable is declared inside the parallel region and is therefore *private* to each thread by default.

2. Exercise 2 – Parallel PI Calculation (Parallel Construct)

2.1 Objective

Parallelise the numerical integration of π using a `#pragma omp parallel` construct *without* using `#pragma omp parallel for`. Pay close attention to shared vs. private variables and measure wall-clock time with `omp_get_wtime()`.

2.2 Implementation

Each thread computes a partial sum over its own subset of the iterations. The partial sums are accumulated into the global `sum` variable with a `critical` section to avoid data races.

```c
#include <stdio.h>
#include <omp.h>

static long num_steps = 100000;
double step;

int main() {
    int i;
    double x, pi, sum = 0.0;
    step = 1.0 / (double)num_steps;

    double t_start = omp_get_wtime();

    #pragma omp parallel private(i, x) shared(sum)
    {
        double local_sum = 0.0;
        int nthreads = omp_get_num_threads();
        int tid      = omp_get_thread_num();

        for (i = tid; i < num_steps; i += nthreads) {
```

```
            x = (i + 0.5) * step;
            local_sum += 4.0 / (1.0 + x * x);
        }

        #pragma omp critical
        {
            sum += local_sum;
        }
    }

    pi = step * sum;
    double t_end = omp_get_wtime();

    printf("PI     = %.5f\n", pi);
    printf("Time   = %f seconds\n", t_end - t_start);
    return 0;
}
```

Listing 4: Exercise 2 – PI with parallel construct

2.3 Output and Analysis

```
PI    = 3.14159
Time  = 0.000460 seconds
```

Listing 5: Output of Exercise 2

- **Result:** $\pi \approx 3.14159$, which matches the known value to 5 significant figures with $N = 100\,000$ steps.
- **Shared vs. private:** The loop index `i` and the intermediate value `x` are declared `private` so that each thread works on its own copy. The global accumulator `sum` is `shared`. To avoid race conditions on `sum`, each thread first accumulates into a thread-local variable (`local_sum`) and then adds to `sum` inside a `critical` section.
- **Timing:** `omp_get_wtime()` measures wall-clock time, making it suitable for measuring parallel speedup.

3. Exercise 3 – PI with Loop Construct

3.1 Objective

Re-parallelise the serial PI program using a `#pragma omp parallel for` loop construct, adding only **one** line to the serial code.

3.2 Implementation

```
#include <stdio.h>
#include <omp.h>

static long num_steps = 100000;
double step;
```

```c

int main() {
    int i;
    double x, pi, sum = 0.0;
    step = 1.0 / (double)num_steps;

    double t_start = omp_get_wtime();

    // Only ONE line added to the serial version:
    #pragma omp parallel for reduction(+:sum) private(x)
    for (i = 0; i < num_steps; i++) {
        x = (i + 0.5) * step;
        sum = sum + 4.0 / (1.0 + x * x);
    }

    pi = step * sum;
    double t_end = omp_get_wtime();

    printf("PI     = %.5f\n", pi);
    printf("Time: %f seconds\n", t_end - t_start);
    return 0;
}
```

Listing 6: Exercise 3 – PI with parallel for (one added line)

3.3 Output and Analysis

```
PI    = 3.14159
Time: 0.000518 seconds
```

Listing 7: Output of Exercise 3

- **Single change:** The only modification to the serial code is the single `#pragma omp parallel for reduction(+:sum) private(x)` directive.
- **Reduction clause:** The `reduction(+:sum)` clause handles the race condition on `sum` automatically.
- **Comparison with Exercise 2:** The loop construct is more concise and achieves the same result. The slightly higher time (0.000518 vs 0.000460 s) is within measurement noise for such short runs; both approaches are equivalent in practice.

4. Exercise 4 – Parallel Matrix Multiplication

4.1 Objective

Parallelise the matrix multiplication kernel with OpenMP directives, use the `collapse` clause, measure speedup and efficiency for 1–16 threads, and compare STATIC, DYNAMIC, and GUIDED scheduling policies with varying chunk sizes.

4.2 Implementation

```c
#include <stdio.h>
#include <stdlib.h>
#include <omp.h>

int main() {
    int m = 500, n = 500;

    double *a = (double *)malloc(m * n * sizeof(double));
    double *b = (double *)malloc(n * m * sizeof(double));
    double *c = (double *)malloc(m * m * sizeof(double));

    // Initialize matrices
    for (int i = 0; i < m; i++)
        for (int j = 0; j < n; j++)
            a[i * n + j] = (i + 1) + (j + 1);

    for (int i = 0; i < n; i++)
        for (int j = 0; j < m; j++)
            b[i * m + j] = (i + 1) - (j + 1);

    for (int i = 0; i < m; i++)
        for (int j = 0; j < m; j++)
            c[i * m + j] = 0;

    double t_start = omp_get_wtime();

    // Parallel matrix multiplication with collapse(2)
    #pragma omp parallel for collapse(2) schedule(guided)
    for (int i = 0; i < m; i++) {
        for (int j = 0; j < m; j++) {
            for (int k = 0; k < n; k++) {
                c[i * m + j] += a[i * n + k] * b[k * m + j];
            }
        }
    }

    double t_end = omp_get_wtime();
    printf("Time = %f seconds\n", t_end - t_start);

    free(a); free(b); free(c);
    return 0;
}
```

Listing 8: Exercise 4 – Matrix multiplication with collapse and scheduling

4.3 Raw Performance Data

4.3.1 Thread Count Scaling (Default Scheduling)

Table 1: Execution time vs. thread count (matrix multiplication, $m = n = 500$)

Threads	Time (s)	Speedup	Efficiency (%)
1	0.269674	1.00	100.0
2	0.121995	2.21	110.5
4	0.067952	3.97	99.2
8	0.052612	5.13	64.1
16	0.049701	5.43	33.9

4.3.2 Scheduling Policy Comparison (4 Threads)

Table 2: Scheduling policy comparison – 4 threads

Policy	Chunk Size	Time (s)
STATIC	1	0.070388
STATIC	10	0.069124
STATIC	50	0.072838
STATIC	100	0.075131
DYNAMIC	1	0.078166
DYNAMIC	10	0.069605
DYNAMIC	50	0.068521
DYNAMIC	100	0.067945
GUIDED	1	0.067717
GUIDED	10	0.067435
GUIDED	50	0.067936
GUIDED	100	0.067579

4.4 Speedup and Efficiency Charts

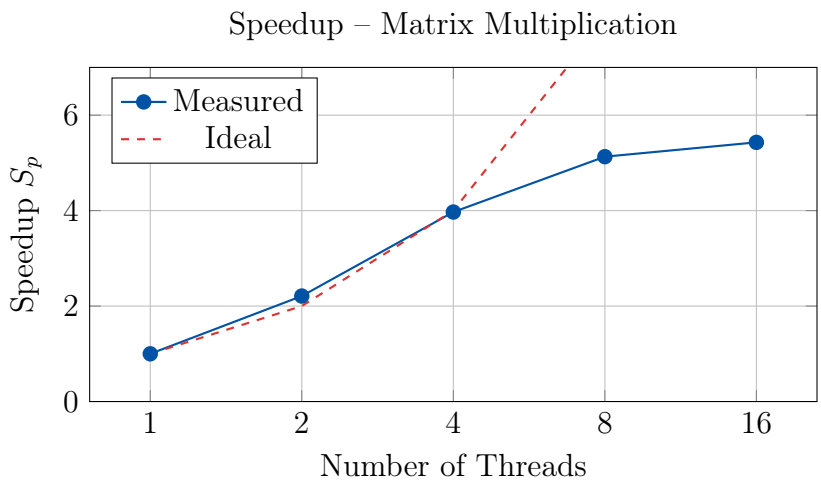

Figure 1: Measured vs. ideal speedup for matrix multiplication

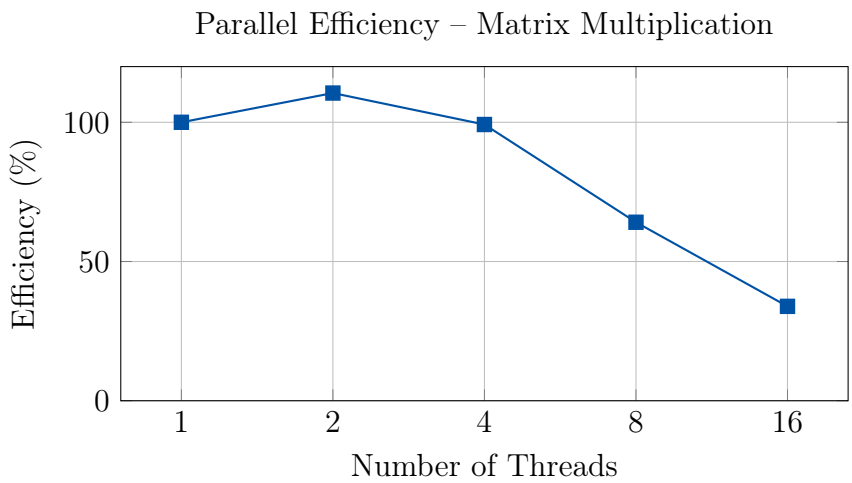

Figure 2: Parallel efficiency for matrix multiplication

4.5 Discussion

4.5.1 Speedup Analysis

- **1 to 4 threads:** Speedup scales well, reaching ≈3.97× with 4 threads. This suggests the workload is well-balanced and the parallelisation overhead is negligible at this scale.
- **8 threads:** Speedup is 5.13× but efficiency drops to 64%. The matrix size (500 × 500) is no longer large enough to keep all 8 threads fully busy relative to the synchronisation and scheduling overhead.
- **16 threads:** Diminishing returns are pronounced; speedup stagnates at 5.43× with only 34% efficiency. This is consistent with Amdahl's Law.

4.5.2 Scheduling Policy Analysis

- **STATIC:** Best with small chunk sizes. Large chunks (100) introduce load imbalance.
- **DYNAMIC:** Performs poorly with chunk size 1 due to high synchronisation overhead. Performance improves with larger chunks.
- **GUIDED:** Consistently delivers the best performance across all chunk sizes. It starts with large chunks and decreases them progressively, naturally balancing load while minimising overhead.
- **Collapse(2):** Flattening the two outer loops into a single iteration space of size $m \times m = 250\,000$ gives more flexibility in distributing work.

5. Exercise 5 – Parallel Jacobi Method

5.1 Objective

Parallelise the Jacobi iterative solver for a linear system $Ax = b$ using OpenMP, and analyse speedup and efficiency for 1–16 threads as well as scalability with increasing system size.

5.2 Parallelisation Strategy

The Jacobi iteration updates every component $x_i^{(k+1)}$ independently using values from the *previous* iteration $x^{(k)}$, making the inner loops over i embarrassingly parallel. The convergence norm computation is parallelised with a `reduction`.

Key directives added:

- `#pragma omp parallel for` on the outer loop (lines 60–69), with `i` and `j` private.
- `#pragma omp parallel for reduction(max:absmax)` on the norm loop (lines 72–76).

5.3 Raw Performance Data

5.3.1 Thread Count Scaling ($n = 120$, diag $= 80$)

Table 3: Jacobi solver – time and convergence vs. thread count

Threads	Time (s)	Speedup	Efficiency (%)	Iterations	Norm
1	7.110E-04	1.00	100.0	10	4.185E-07
2	4.940E-04	1.44	72.0	10	4.434E-07
4	3.240E-04	2.19	54.8	12	1.994E-07
8	6.730E-04	1.06	13.2	11	3.009E-07
16	2.627E-03	0.27	1.7	16	5.696E-07

5.3.2 System Size Scaling (4 Threads)

Table 4: Jacobi solver – scalability with system size (4 threads)

System size n	Time (s)	Iterations	Norm
120	4.900E-04	13	3.945E-07
240	1.014E-03	15	7.131E-07
480	6.089E-03	30	9.002E-07
960	6.421E-02	171	8.993E-07

5.4 Speedup and Efficiency Charts

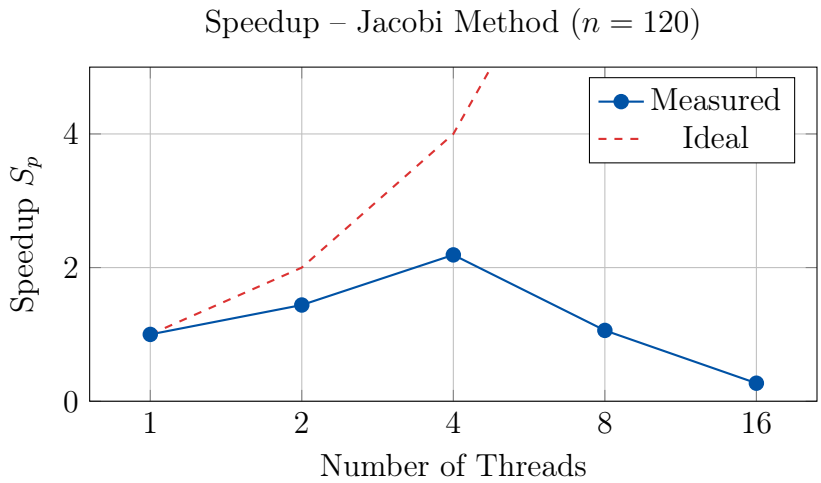

Figure 3: Measured vs. ideal speedup for the Jacobi solver

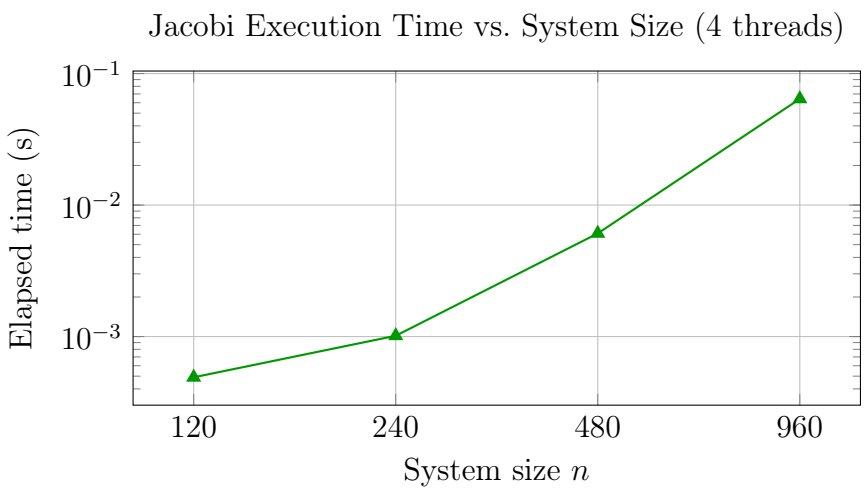

Figure 4: Execution time vs. system size for the Jacobi solver (log-log scale)

5.5 Discussion

5.5.1 Thread Scaling ($n = 120$)

The Jacobi results reveal a **poor parallel efficiency** for this small problem size, for the following reasons:

- **Problem too small:** With $n = 120$, the outer loop has only 120 iterations. The overhead of creating and synchronising threads quickly dominates the useful computation.
- **8 threads:** Speedup collapses to ≈1.06, meaning essentially no benefit over sequential execution. Thread creation and barrier overhead consume most of the elapsed time.
- **16 threads:** Speedup is 0.27, meaning the parallel version is **3.7× slower** than serial. This is over-parallelisation.
- **Variation in iteration count:** The number of Jacobi iterations varies slightly across thread counts. This is caused by the following: the order in which partial sums are accumulated changes with the number of threads, thus altering convergence behaviour.

5.5.2 System Size Scaling (4 Threads)

- The elapsed time grows super-linearly with n. Doubling n from 120 to 240 roughly doubles the time (0.49 ms → 1.01 ms), but from 480 to 960 the time increases by a factor of ≈10.5.
- The Jacobi method converges slowly for large, less diagonally-dominant systems.
- For larger sizes ($n \geq 480$) the parallel version with 4 threads would offer meaningful speedup.